

PRD: recipe4fridge_pic

v0 — 최초 초안 (Notion에 저장해둔 원본)

1. 개요

냉장고 사진을 업로드하면 AI가 식재료를 인식하고, 사용자의 선호 조건을 반영해 레시피를 추천하는 웹 앱.

- **비전 AI**: 사진 → 식재료 종류/양 추정
- **텍스트 AI**: 식재료 + 선호 조건 → 레시피 추천
- **인증**: 회원가입/로그인 후 이용, 선호 조건과 추천 기록을 계정에 저장

2. 목표와 범위

구분	포함
목표	사진 한 장으로 "오늘 뭐 해먹지"에 대한 실용적 답을 빠르게 제공
비목표 (초기 범위 제외)	정확한 칼로리 계산, 재고 관리(유통기한 알림 등), 다중 사진 업로드, 모바일 네이티브 앱

3. 기술 스택

- **프론트/백엔드**: Next.js (App Router), Vercel 배포
- **인증/DB/스토리지**: Supabase (Auth, Postgres, Storage)
- **AI 연동**: OpenRouter 등 무료 API 제공처, 비전/텍스트 모델 각각 사용자가 선택 가능한 구조
- **CI/CD**: GitHub 저장소 연동 + Vercel 자동 배포

4. 사용자 흐름

1. **가입/로그인** — Google OAuth 또는 이메일/비밀번호 (Supabase Auth)
2. **선호 조건 설정** (최초 1회, 이후 마이페이지에서 수정) — 매운맛 선호도, 조리 난이도, 조리 소요 시간 등
3. **냉장고 사진 업로드** — 이미지 1장 업로드
4. **식재료 인식 결과 확인** — 인식된 식재료 목록(종류/수량 추정치) 표시, 사용자가 추가/수정/삭제 가능
5. **레시피 추천 요청** — 확정된 식재료 + 선호 조건으로 레시피 N개 추천받음
6. **레시피 상세 확인 및 저장** — 마음에 드는 레시피를 계정에 저장, 이후 "추천받았던 레시피" 목록에서 재확인

5. 기능 요구사항

5.1 인증 및 회원 관리

- Google OAuth 로그인
- 이메일/비밀번호 회원가입, 로그인, 로그아웃
- 비밀번호는 Supabase Auth가 관리 (직접 저장/해싱하지 않음)

5.2 선호 조건 (사용자 프로필)

- 매운맛 선호도 (예: 안 매움/보통/매움)
- 조리 난이도 선호 (쉬움/보통/어려움)
- 조리 소요 시간 선호 (예: 15분 이내/30분 이내/제한 없음)
- 디자인 테마 선택 (2~3종 중 택1)

5.3 식재료 인식

- 이미지 업로드 (jpg/png, 용량 제한 명시 필요 — 예: 10MB 이하)
- 선택된 비전 AI API 호출 → 식재료명 + 추정 수량 목록 반환
- 인식 결과에 대한 수동 추가/수정/삭제 UI
- 비전 API 제공처를 설정 화면에서 선택 가능 (API별 특성/한도 간단 안내 포함)

5.4 레시피 추천

- 확정된 식재료 목록 + 선호 조건을 프롬프트로 구성해 텍스트 AI 호출
- 레시피 결과: 요리명, 필요 재료, 조리 단계, 예상 소요 시간 포함
- 텍스트 AI API 제공처를 설정 화면에서 선택 가능
- 추천 결과 저장 (즐거찾기), 저장 목록 조회/삭제

5.5 디자인 테마

- 기본 테마: 밝고 친숙한 톤
- 사용자별 2~3개 테마 중 선택 가능 (라이트 게일 변형)
- 테마 시안은 구현 전 별도 검토/확정 단계 필요

6. 데이터 모델 (초안)

```
users          (Supabase Auth 기본 제공 테이블 사용)
profiles       id(=user_id, FK), spice_level, difficulty, time_limit, theme, created_at
fridge_uploads id, user_id(FK), image_url, vision_provider, created_at
detected_ingredients id, upload_id(FK), name, quantity_text, is_user_edited
recipe_requests id, user_id(FK), upload_id(FK), text_provider, created_at
recipes         id, request_id(FK), title, ingredients_json, steps_json, est_time_minutes
saved_recipes   id, user_id(FK), recipe_id(FK), saved_at
```

7. API / AI 연동 설계

- **추상화 계층**: VisionProvider / TextProvider 인터페이스를 두고, 각 무료 API(OpenRouter 경유 모델 등)를 구현체로 연결
- 예: OpenRouterVisionProvider, OpenRouterTextProvider — 이후 다른 무료 제공처 추가 시 인터페이스만 구현하면 됨
- **API 키 관리**: 서버 환경변수로 보관, 클라이언트에 절대 노출 금지
- **요청 흐름**: 클라이언트 → Next.js API Route(서버) → 선택된 Provider 호출 → 결과 정규화 후 응답
- **실패 처리**: 무료 API 한도 초과/오류 시 사용자에게 명확한 안내 + 다른 제공처 재시도 유도

8. 비기능 요구사항

8.1 보안

- API 키 등 민감정보는 서버 사이드에서만 사용 (env 변수, 클라이언트 번들에 포함 금지)
- Supabase RLS(Row Level Security)로 사용자 데이터 접근 제어 (자신의 데이터만 조회/수정 가능)
- 업로드 이미지 용량/형식 검증 (서버 사이드 재검증)
- 인증 토큰 만료/재발급 처리
- 주요 위협 요소 점검 목록: 인증 우회, 이미지 업로드를 통한 악성 파일 삽입, API 키 노출, 과도한 API 호출로 인한 비용/한도 남용(rate limiting)

8.2 성능/비용

- 무료 API 티어 한도 내에서 동작하도록 호출 횟수 최소화 (같은 이미지 재분석 방지 등 캐싱 고려)

8.3 접근성/반응형

- 모바일/데스크톱 반응형 레이아웃
- 기본 대비/폰트 크기 등 접근성 고려

9. 마일스톤

1. PRD 확정 (현재 단계)
2. 디자인 시안 2~3종 확정
3. MVP: 인증 없이 로컬에서 사진 업로드 → 식재료 인식 → 레시피 추천 파이프라인 동작 확인
4. 인증 + 개인화: 로그인, 선호 조건, 저장 기능 연결
5. API 다중 선택 UI: 비전/텍스트 Provider 선택 화면 구현
6. 보안 점검 및 대응 구현
7. 배포: GitHub 연동 + Vercel 배포
8. 문서화: 기획/설계/개발/보완 과정 정리, 강의용 도식 포함

10. 미결정/확인 필요 사항

- 비전/텍스트 각각 초기 지원할 무료 API 제공처 목록 (OpenRouter 내 구체적 모델 후보 선정 필요)
- 이미지 업로드 용량/개수 제한 구체 수치
- 디자인 테마 3종의 구체적 컨셉 (색상/톤)
- 레시피 추천 개수 (기본 N=3 등) 및 재요청(다른 레시피 더 보기) 지원 여부